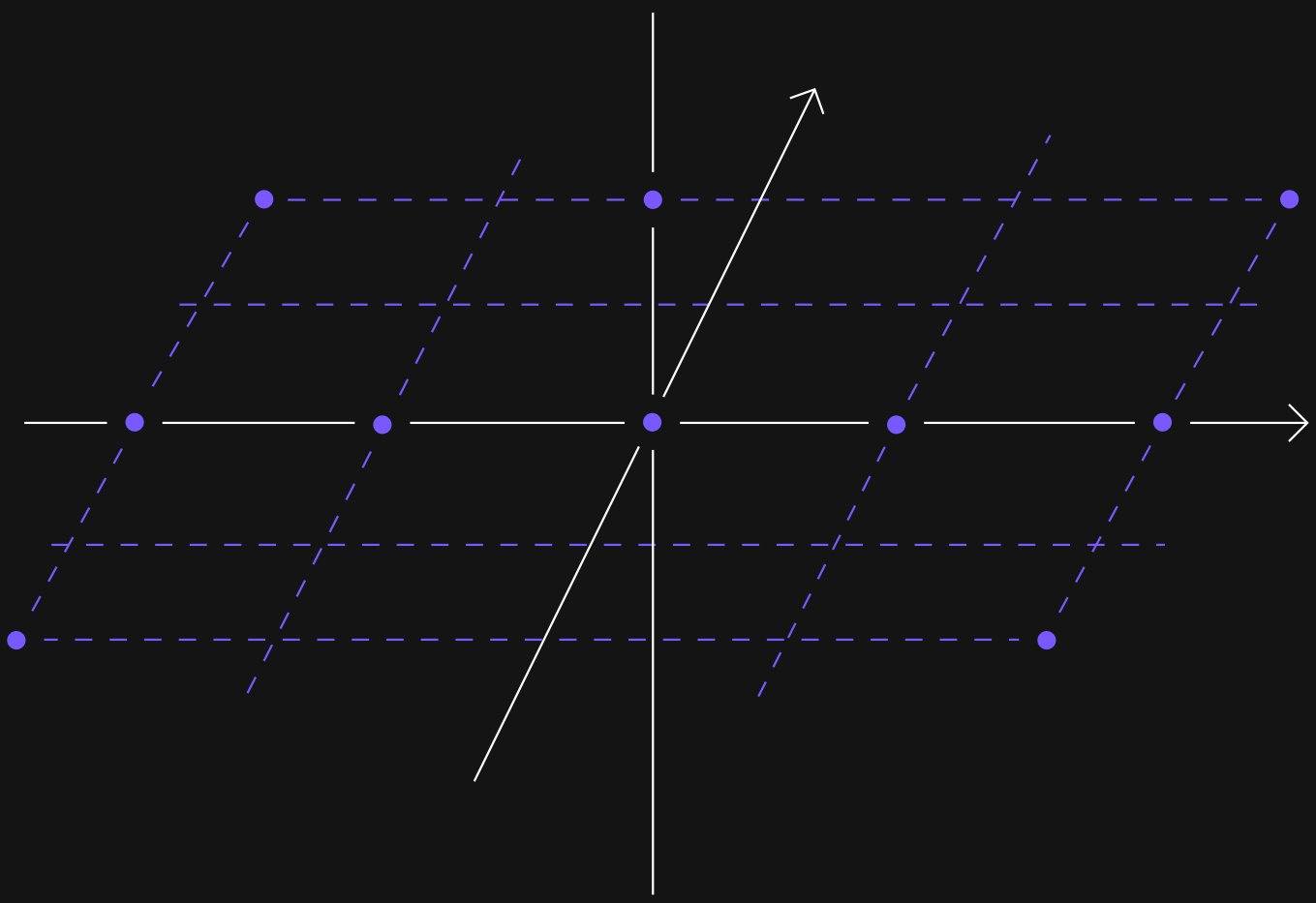
Разработка

Лонгрид

Сетевые технологии

03

Стек протоколов. Транспортный уровень



СЕТЕВЫЕ ТЕХНОЛОГИИ · ЛЕКЦИЯ 3 · ЛОНГРИД

Транспортный уровень: доставка данных, соединения и сеансы

Доставка данных, соединения и сеансы. TCP, UDP, SCTP и DCCP; справочник RPC и протоколов связи.

Материалы курса · запуск 2026

Содержание

01 Введение

02 Порты и TCP-соединения

03 UDP: передача отдельных датаграмм

04 TCP: упорядоченный поток байтов

05 Как устанавливается TCP-соединение

06 Потери, перестановки и повторная передача

07 Как закрывается TCP-соединение

08 Как получатель и сеть ограничивают скорость передачи

09 Поля и дополнительные механизмы TCP — справочник

10 SCTP и DCCP

11 RPC и удалённые операции — справочник

12 Сигнализация и передача звука — справочник

13 H.323, SIP и распределение функций — справочник

14 Безопасность и переход к QUIC

15 Самопроверка

16 Источники и материалы курса

1. Введение

В первой лекции мы проследили путь запроса к сервису. Во второй разобрали, как приложения читают сообщения: находят их границы, разбирают формат данных и используют TLS для защиты обмена. Теперь посмотрим, как данные передаются между приложениями.

Почему передача может замедлиться или оборваться, даже если с самим сообщением всё в порядке? Чтобы ответить на этот вопрос, нужно понять, что делает транспортный протокол и какие задачи остаются приложению.

OSI	Задача	Связь с курсом
L7, прикладной	Обмен запросами и ответами по правилам приложения	HTTP, DNS и почтовые протоколы из лекции 1
L6, представления	Формат данных и их преобразование	MIME, кодировки, сериализация и защита из лекции 2
L5, сеансовый	Организация диалога между приложениями	В этой лекции — на примерах RPC и телефонии
L4, транспортный	Передача данных между приложениями	Основная тема этой лекции
L3, сетевой	Доставка пакетов между узлами	Пока используем IP, маршрутизацию разберём позже
L2, канальный	Передача данных между соседними узлами	Разберём далее
L1, физический	Передача сигналов	Разберём далее

Модель OSI помогает разделить задачи. В реальной системе один протокол или одна библиотека могут выполнять несколько из них. Сначала мы изучим транспорт, а затем вернёмся к организации диалога между приложениями.

Для HTTPS поверх TCP цепочка выглядит так: HTTP → TLS → TCP → IP. У HTTP/3 она другая: HTTP/3 → QUIC → UDP → IP. QUIC подробно рассмотрим в следующей лекции. Выбор транспорта связан с требованиями приложения: при передаче файла важно получить все данные, а во время разговора опоздавший фрагмент звука может уже не понадобиться.

2. Порты и TCP-соединения

IP-адрес позволяет направить пакет к узлу, но на одном узле работает много сетевых приложений. TCP и UDP используют номера портов, чтобы различать обмены этих приложений. В обычном случае приложение отправляет и принимает данные через сокет, который обслуживает сетевой стек ОС.

Сокет — объект, с которым приложение работает через socket API. Номер порта — поле в заголовке протокола. При этом понятие порта есть не у всех сетевых протоколов; такие случаи рассмотрим в следующих лекциях.

Сервер может слушать TCP-порт 443 и одновременно обслуживать несколько соединений:

```
клиент A: 192.0.2.10:53000    → сервер: 198.51.100.20:443  TCP
клиент B: 192.0.2.11:53000    → сервер: 198.51.100.20:443  TCP
клиент A: 192.0.2.10:53001    → сервер: 198.51.100.20:443  TCP
```

Каждая строка обозначает отдельное TCP-соединение. Его определяют две пары `IP:port`: адрес и порт клиента, адрес и порт сервера. Если добавить название транспортного протокола, получится набор из пяти полей: source IP, destination IP, source port, destination port, protocol. Его часто называют **5-tuple** и используют для идентификации конкретной транспортной сессии. В ответных пакетах адреса и порты меняются местами, но это тот же двусторонний обмен.

В примере оба клиента используют порт 53000. Это допустимо: у них разные IP-адреса. TCP и UDP также используют номера портов независимо друг от друга, поэтому один узел может принимать TCP- и UDP-трафик на порту 53. Исходный порт клиента обычно выбирает ОС из динамического диапазона. Границы диапазона и правила выбора зависят от системы и её настроек.

Номер порта подсказывает, какой сервис мы ожидаем увидеть, но не гарантирует использование определённого протокола. HTTPS может работать на нестандартном порту, а на порту 443 может отвечать другой сервис.

Одно приложение может использовать несколько сокетов. Например, прокси принимает TCP-соединение от клиента и открывает отдельное соединение с сервером. TCP доставляет данные между участниками каждого из этих соединений; за передачу запроса дальше отвечает прокси.

NAT может изменять IP-адреса и номера портов, поэтому при наблюдении пакетов до и после NAT один и тот же обмен может выглядеть как разные наборы адресов и портов. Подробнее разберём NAT в одной из следующих лекций.

3. UDP: передача отдельных датаграмм

UDP — User Datagram Protocol — передаёт отдельные сообщения, которые называют датаграммами. Приложение может отправить датаграмму, не устанавливая соединение с получателем. Например, при обычном DNS-запросе клиент отправляет датаграмму с вопросом, а сервер возвращает датаграмму с ответом. Как мы уже знаем, DNS может использовать и другие транспорты.

Заголовок и свойства

Заголовок UDP занимает 8 байт и содержит четыре поля:

Поле	Назначение
Source Port	Порт отправителя
Destination Port	Порт получателя
Length	Общая длина заголовка UDP и данных
Checksum	Контрольная сумма для обнаружения ошибок при передаче

UDP сохраняет границы датаграмм: две отправленные датаграммы остаются двумя отдельными сообщениями. Однако они могут потеряться, продублироваться или прийти в другом порядке. Сам UDP не повторяет потерянные датаграммы и не требует подтверждать их получение. Если приложению нужны подтверждения или повторы, оно должно организовать их само либо использовать протокол поверх UDP, который это делает.

Контрольная сумма помогает обнаружить случайное повреждение данных. Она учитывает заголовок UDP, данные и псевдозаголовок IP — набор сведений из IP, в том числе адреса отправителя и получателя. Криптографической защиты контрольная сумма не даёт. В IPv4 нулевое значение поля Checksum означает, что проверка не используется. В IPv6 контрольная сумма обычно обязательна; исключения предусмотрены для отдельных туннельных механизмов. Подробности и рекомендации для приложений: RFC 8085.

Что делает connect() для UDP

В socket API можно вызвать `connect()` и для UDP-сокета. В этом случае ОС связывает сокет с адресом и портом выбранного собеседника: их не нужно указывать при каждой отправке, а приём на этом сокете ограничивается датаграммами от него. Для приложения это удобная абстракция ОС, через которую оно отправляет и получает данные по UDP.

В отличие от TCP, такой вызов не запускает handshake с удалённой стороной и не подтверждает, что она доступна. Данные по-прежнему передаются отдельными датаграммами: потока байтов, как в TCP, не возникает. Пример поведения этого API в Linux описан в udp(7) и connect(2).

Почему приложения выбирают UDP

У звука во время разговора есть срок годности: фрагмент, который пришёл после нужного момента воспроизведения, может быть отброшен. Поэтому получателю важны потери, задержка и jitter (джиттер — изменение задержки доставки от пакета к пакету). Например, один пакет дошёл за 20 мс, следующий — за 35 мс, а третий — за 22 мс. Равномерно отправленные пакеты поступают неравномерно.

Приложение может временно накапливать данные в буфере, чтобы сгладить эти колебания, восстанавливать часть потерь или снижать качество кодирования. UDP позволяет приложению выбрать подходящие механизмы, но сам их не предоставляет.

Управления перегрузкой в UDP тоже нет. Если оно требуется для такого обмена, скорость отправки регулирует приложение или протокол, работающий поверх UDP.

4. TCP: упорядоченный поток байтов

TCP — Transmission Control Protocol — передаёт данные как поток байтов. В TCP-соединении обе стороны могут отправлять данные. В каждом направлении получатель читает байты в том порядке, в котором их отправили. Если сегменты пришли не по порядку или продублировались, TCP обрабатывает это до передачи данных приложению.

Если часть данных потерялась, TCP пытается передать её повторно. На это требуется время, поэтому приложение может заметить паузу. Если продолжить обмен не удаётся, соединение может завершиться ошибкой. Таким образом, надёжная доставка в TCP не означает, что любая передача обязательно завершится успешно или уложится в тайм-аут приложения. Основная спецификация: RFC 9293.

Как приложение находит границы сообщений

Предположим, приложение отправляет две строки, каждая заканчивается LF:

```
PING\nPONG\n
```

Получатель может прочитать сначала `PING\n`, затем `PONG\n`. Он может получить `PI`, затем `NG\nPO`, затем `NG\n` или обе строки за одно чтение. Во всех случаях последовательность из десяти байтов одинакова.

TCP сохраняет порядок байтов, но не границы отдельных отправок. Поэтому приложение само определяет, где заканчивается сообщение: по разделителю, фиксированной длине, длине в заголовке или правилам формата. В лекции 2 мы встречали Content-Length и chunked для HTTP/1.1 — они решают именно эту задачу. Если приложение прочитало меньше байтов, чем ожидало, это ещё не означает конец потока: остальные данные могут прийти позже.

На семинаре мы ограничим размер буфера чтения и посмотрим, как строка разбивается на части. Размер прочитанного блока при этом не показывает размер TCP-сегмента. ОС использует свои буферы и определяет, как разделить байты на сегменты: два вызова отправки не обязательно создают два сегмента.

Сегментация и размер полезных данных

TCP делит поток байтов на сегменты, которые передаются внутри IP-пакетов. MTU — Maximum Transmission Unit — ограничивает размер IP-пакета, который можно передать по соответствующему каналу без фрагментации. MSS — Maximum Segment Size — задаёт максимальный размер данных TCP в сегменте, без заголовков TCP и IP. Значение MSS стороны сообщают при установлении соединения.

Например, при MTU 1500 байт, заголовке IPv4 длиной 20 байт и заголовке TCP длиной 20 байт для данных остаётся 1460 байт. Опции в заголовках, другая версия IP или туннель меняют этот расчёт. Короткий сегмент также допустим: TCP не обязан каждый раз заполнять его до максимального размера.

TCP-сегментация и IP-фрагментация работают с разными объектами. TCP делит поток байтов на сегменты, а при IP-фрагментации один IP-пакет разбивается на фрагменты. Возможность фрагментации и её правила зависят от версии IP. Выбор размера TCP-сегментов с учётом MTU помогает её избежать.

Path MTU Discovery, PMTUD, позволяет определить размер IP-пакета, который пройдёт по всему пути без фрагментации. Это тема сетевого уровня; подробнее разберём её позже. Классический механизм для IPv4 описан в RFC 1191.

Как устроен заголовок TCP

Каждый сегмент начинается с заголовка. В нём указаны порты, номера байтов, подтверждение, флаги и окно получателя. Обязательная часть занимает 20 байт — первые пять строк на схеме. После неё могут идти опции.

Заголовок TCP: каждая из первых пяти строк занимает 32 бита

<table>
<tr><td colspan="3">Порт отправителя — 16 бит</td><td>Порт получателя — 16 бит</td></tr>
<tr><td colspan="4">Номер последовательности SEQ — 32 бита</td></tr>
<tr><td colspan="4">Номер подтверждения ACK — 32 бита</td></tr>
<tr><td>Длина заголовка — 4</td><td>Резерв — 4</td><td>Флаги — 8</td><td>Окно получателя — 16</td></tr>
<tr><td colspan="3">Контрольная сумма — 16 бит</td><td>Указатель срочных данных — 16 бит</td></tr>
<tr><td colspan="4">Опции и выравнивание — от 0 до 40 байт</td></tr>
</table>

Поле Data Offset задаёт длину заголовка в 32-битных словах. Значение 5 означает 20 байт, значение 8 — 32 байта, а максимальное значение 15 — 60 байт. По этому полю получатель определяет, где начинаются данные. MSS, SACK и Window Scale относятся к опциям и не входят в обязательные 20 байт.

Обозначение ACK встречается в двух местах: так называют и флаг, и поле номера подтверждения. Установленный флаг ACK сообщает, что номер подтверждения нужно учитывать. ISN — начальное значение номера последовательности SEQ; отдельного поля ISN нет. Окно перегрузки cwnd хранится у отправителя и в заголовок не входит.

Назначение полей и флагов подробнее разобрано в разделе 9. Формат заголовка: RFC 9293, §3.1.

Нумерация байтов и подтверждения

```
Учебный обмен, одно направление данных
Клиент → сервер: SEQ=1001  LEN=500   [байты 1001 ... 1500]
Сервер → клиент: ACK=1501           [ожидаю следующий байт 1501]
```

SEQ указывает номер первого байта данных в сегменте. ACK — Acknowledgment — содержит номер следующего ожидаемого байта от другой стороны. Для каждого направления соединения TCP ведёт собственную нумерацию.

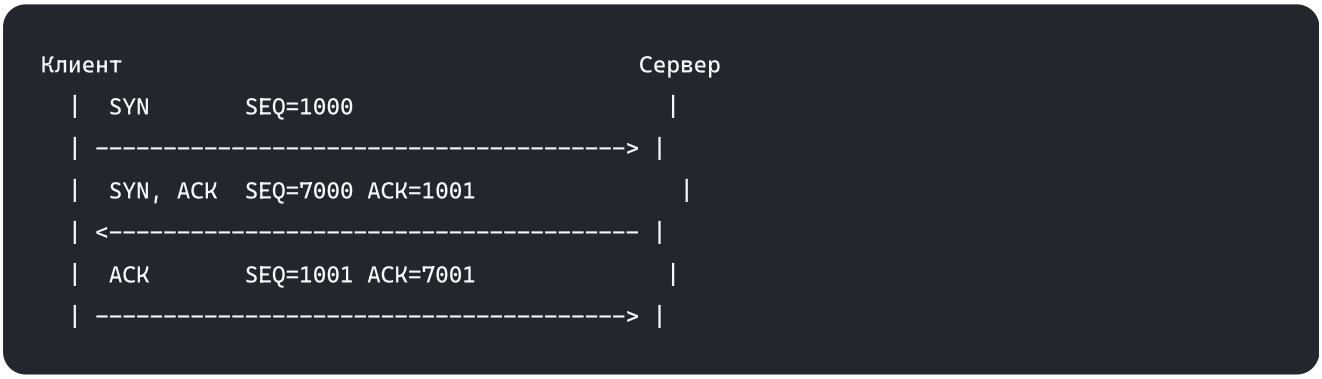
ACK подтверждает все полученные подряд байты. В нашем примере ACK=1501 означает, что TCP получил все байты до 1500 включительно и теперь ждёт байт 1501. Такое подтверждение называют **кумулятивным**.

Подтверждение отправляет сетевой стек получателя. Оно ещё не означает, что приложение прочитало данные, выполнило запрос или сохранило результат. Поэтому у HTTP, SMTP и RPC есть собственные ответы.

В Wireshark номера часто показаны относительно начала соединения. Ниже разберём, чем такое отображение отличается от значений в самих пакетах.

5. Как устанавливается TCP-соединение

Перед передачей данных стороны согласуют начальные номера последовательности и параметры соединения. Начальный номер называется ISN — Initial Sequence Number. Для согласования используется флаг SYN — Synchronize. Обычное установление соединения состоит из трёх шагов:



Клиент начинает с SYN и сообщает свой ISN. Сервер отвечает SYN+ACK: сообщает собственный ISN и подтверждает SYN клиента. Затем клиент подтверждает SYN сервера. SYN занимает один номер в последовательности, хотя не содержит байта пользовательских данных. Поэтому подтверждения в примере равны 1001 и 7001.

После этого стороны могут передавать данные приложения. Для HTTPS поверх TCP следующим этапом будет TLS handshake. При поиске причин задержки важно измерять эти этапы отдельно: ожидание TCP-соединения и согласование TLS могут занимать разное время.

Откуда берётся ISN

Начальный номер не обязан быть нулём или единицей. В ранней спецификации TCP предлагалось выбирать его с помощью растущего счётчика, связанного со временем. Это помогало отличать новое соединение от задержавшихся сегментов предыдущего соединения с теми же адресами и портами.

Однако предсказуемые номера позволяли атакующему пытаться подделывать сегменты, даже не наблюдая обмен. Позднее в выбор ISN добавили псевдослучайную составляющую, которая зависит от адресов, портов и секрета узла. Зная номера одного соединения, не должно быть легко угадать номера другого. История и алгоритм описаны в RFC 6528, а рекомендации включены в RFC 9293, §3.4.1. Атаки на TCP подробнее разберём в лекции о защите.

Wireshark часто показывает **относительные номера**: вычитает ISN соответствующего направления. Тогда у SYN видно SEQ=0, а первый байт данных имеет SEQ=1. Реальный ISN в пакете при этом может быть большим и непредсказуемым. В наших примерах значения 1000 и 7000 выбраны для удобства расчёта.

Что можно узнать по пакетам

Предположим, мы записали трафик на клиенте и открыли его в Wireshark. Каждое наблюдение позволяет сделать определённый вывод, но не объясняет сразу всё, что произошло на сервере.

Что видим	Что это означает	Чего мы пока не знаем
SYN, SYN+ACK, ACK	Клиент отправил завершающий ACK трёхстороннего обмена	Получил ли его сервер и готово ли приложение обработать запрос
После SYN пришёл RST	На попытку подключения пришёл отказ	Ответил сам сервер или промежуточное устройство
Клиент повторяет SYN, ответа нет	В записи на клиенте нет ответа	Дошёл ли SYN до сервера и был ли отправлен ответ
ACK подтверждает байты запроса	TCP на принимающей стороне получил эти байты	Прочитало ли их приложение и выполнило ли операцию

Успешный вызов `send()` означает, что приложение передало данные сетевому стеку ОС. Возвращаемое значение показывает, сколько байтов принято для отправки. Оно может быть меньше размера переданного блока — тогда приложению нужно отправить остаток по правилам API. Успех `send()` ещё не подтверждает доставку другой стороне.

Далее данные должны пройти по сети, попасть в сетевой стек получателя и быть прочитаны приложением. Если запрос меняет данные в хранилище, появляется ещё один этап — выполнение и сохранение результата.

Тайм-аут тоже относится к определённому этапу: поиску адреса через DNS, установлению TCP-соединения, TLS handshake, ожиданию ответа или операции целиком. Увеличение одного тайм-аута не обязательно меняет остальные.

6. Потери, перестановки и повторная передача

IP-пакеты могут потеряться, задержаться, продублироваться или прийти не по порядку. TCP восстанавливает порядок байтов и повторно передаёт недостающие данные. Пока это происходит, приложение может ждать.

Что происходит при пропуске байтов

Сервер ждёт байт 1001, а клиент отправляет три диапазона. Предположим, сервер сохраняет данные, которые пришли после пропуска:

Диапазон	Событие	Подтверждение сервера
1001–1500	Получен	ACK=1501
1501–2000	Пока не получен	Сервер продолжает ждать байт 1501
2001–2500	Получен и сохранён в буфере	ACK=1501: байты 1501–2000 всё ещё отсутствуют
1501–2000	Пришёл позднее или был передан повторно	ACK=2501

TCP может передать приложению байты до 1500 включительно. Байты 2001–2500 пока остаются в буфере: передать их приложению можно после получения недостающих байтов 1501–2000. Затем весь диапазон 1501–2500 становится доступен для чтения. Обмен в обратном направлении при этом может продолжаться.

Опция SACK — Selective Acknowledgment — позволяет получателю сообщить, какие диапазоны он уже сохранил после пропуска. Стороны согласуют поддержку SACK при установлении соединения. В нашем примере получатель может сообщить диапазон `[2001, 2501)`: левая граница включена, правая — нет. Так отправитель узнаёт, какие данные повторять не требуется.

Значение обычного ACK остаётся 1501 до заполнения пропуска. SACK дополняет подтверждение, но порядок передачи байтов приложению остаётся прежним. Описание механизма: RFC 2018.

Когда TCP повторяет передачу

Отправитель измеряет RTT — Round-Trip Time, время прохождения данных и подтверждения туда и обратно. На основе этих измерений и их колебаний он рассчитывает RTO — Retransmission Timeout, время ожидания подтверждения перед повторной передачей. Если таймер срабатывает снова, ожидание увеличивается. RTO обычно отличается от RTT; для каждого сегмента не требуется отдельный таймер. Алгоритм описан в RFC 6298.

О пропуске можно узнать и раньше тайм-аута. Когда получатель продолжает принимать данные после пропуска, он может отправлять повторные ACK с тем же номером. Классический механизм fast retransmit запускает повторную передачу после трёх таких подтверждений — duplicate ACK. Современные реализации используют и другие признаки, в том числе время отправки и подтверждения сегментов. Один из примеров — RACK-TLP, описанный в RFC 8985.

Повторная передача показывает, что отправитель заподозрил потерю данных, но сама по себе не доказывает перегрузку сети. Мог потеряться ACK, сегмент мог сильно задержаться или прийти не по порядку. А в неполной записи трафика часть обмена может просто отсутствовать.

Почему потеря в TCP задерживает потоки HTTP/2

В одном TCP-соединении HTTP/2 передаёт фреймы разных прикладных потоков. Представим, что потерян сегмент с данными первого потока, а за ним уже пришли данные второго. TCP не может передать приложению более поздние байты, пока не получит недостающие. В результате второй поток тоже ждёт. Такую блокировку называют **Head-of-Line blocking, HOL**.

В HTTP/1.1 при конвейерной отправке запросов есть другое ограничение: ответы нужно возвращать в порядке запросов. В HTTP/2 этого требования между потоками нет, но зависимость от порядка байтов TCP остаётся. Это различие разобрано в RFC 9113. В следующей лекции посмотрим, как QUIC меняет работу с потоками.

7. Как закрывается TCP-соединение

FIN и закрытие одного направления

Отправляя FIN, сторона сообщает, что больше не будет передавать данные в этом направлении. FIN занимает один номер в последовательности, как и SYN. Другая сторона подтверждает FIN, но может продолжать свою отправку. Такое состояние называют half-close — закрытием одного направления.

```
Клиент → сервер: последние данные, затем FIN
Сервер → клиент: ACK
Сервер → клиент: оставшиеся данные ответа, затем FIN
Клиент → сервер: ACK
```

Это один из возможных вариантов. FIN и ACK могут передаваться вместе с данными, а обе стороны могут начать закрытие одновременно.

Когда приложение прочитает все байты, предшествовавшие FIN, оно получит признак конца потока — EOF. Достаточно ли этих байтов для полного сообщения, определяет прикладной протокол. Например, если HTTP-заголовок обещал тело длиной 1000 байт, а до EOF пришло только 700, сообщение получено не полностью.

RST и сброс соединения

RST — Reset — прерывает соединение. Например, узел может отправить RST, если получил сегмент для соединения, которого у него нет, или приложение аварийно закрыло сокет. RST может сформировать и промежуточное устройство. Чтобы установить причину, нужно посмотреть, когда и с какой стороны пришёл RST, и сопоставить пакеты с журналами обеих сторон.

TCP проверяет, допустим ли входящий RST для текущего соединения. В некоторых случаях вместо немедленного сброса используется дополнительная проверка с challenge ACK. Это затрудняет подделку RST; подробности описаны в RFC 5961. TLS защищает содержимое сообщений, но не гарантирует, что TCP-соединение никогда не прервётся.

TIME_WAIT и CLOSE_WAIT — справочно

После закрытия ОС некоторое время может хранить сведения о TCP-соединении. В состоянии TIME_WAIT она учитывает запоздавшие сегменты и при необходимости повторяет завершающий ACK. Обычно это состояние возникает у стороны, которая первой начала закрытие; одновременное закрытие имеет свои особенности.

Само наличие TIME_WAIT нормально. Проверять проблему стоит, если таких соединений много и они приводят к нехватке ресурсов, например доступных портов.

CLOSE_WAIT означает, что FIN другой стороны уже получен, но местное приложение ещё не завершило свою отправку. Короткое пребывание в этом состоянии допустимо. Если соединений в CLOSE_WAIT становится всё больше и они долго не закрываются, стоит проверить, закрывает ли приложение свои сокеты после завершения обмена.

TCP keepalive, HTTP keep-alive и heartbeat

Сходные названия могут обозначать разные задачи:

Механизм	Для чего нужен	Что важно учитывать
TCP keepalive	Проверить отклик другой стороны после долгого отсутствия обмена	Ответ сетевого стека не показывает, что приложение способно выполнить запрос
HTTP keep-alive / persistent connection	Использовать одно соединение для нескольких запросов и ответов	Сам по себе не проверяет работоспособность сервиса
Прикладной heartbeat	Проверить приложение с помощью специального сообщения	Проверяет только то, что предусмотрено его обработчиком

Параметры TCP keepalive зависят от ОС и приложения. Прокси или NAT могут перестать обслуживать соединение без трафика раньше, чем отправится первая проверка keepalive. Поэтому при настройке нужно учитывать тайм-ауты всех участвующих устройств.

8. Как получатель и сеть ограничивают скорость передачи

Соединение установлено, данные приходят в правильном порядке, но передача идёт медленно. Возможно, приложение-получатель не успевает их читать. Другая возможная причина — сеть: на одном из участков пакеты поступают быстрее, чем их удаётся передавать дальше, и накапливаются в очереди.

В TCP эти задачи решают два механизма. **Управление потоком, flow control,** помогает не перегружать получателя. **Управление перегрузкой, congestion control,** регулирует нагрузку на сеть.

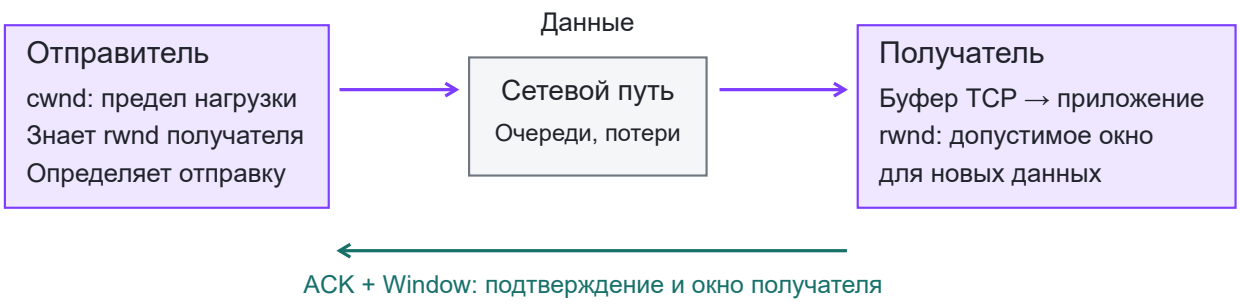


Рисунок 1. Отправитель учитывает возможности и получателя, и сети.

Получатель сообщает своё окно приёма rwnd, а отправитель рассчитывает окно перегрузки cwnd. Отправитель должен соблюдать оба ограничения. Для передачи в обратном направлении действуют свои окна.

Окно получателя: rwnd

Окно получателя, rwnd (receive window), сообщает, какой диапазон байтов получатель готов принять начиная с номера ACK. Принятые данные могут оставаться в буфере TCP, пока приложение их не прочитает. TCP уже может подтвердить их получение, но само подтверждение не освобождает буфер.



Рисунок 2. Буфер получателя, в который помещаются восемь одинаковых блоков.

Сначала в буфере лежат три непрочитанных блока, и ещё пять можно принять. Затем приложение перестаёт читать, новые данные занимают оставшееся место, и получатель объявляет нулевое окно. После чтения четырёх блоков место освобождается: получатель снова может разрешить передачу.

На рисунке показана упрощённая ситуация: данные приходят по порядку, а между этапами нет новых данных, оставшихся в пути. В реальной ОС размер окна зависит от работы сетевого стека и его буферов; он не равен всей свободной оперативной памяти.

При rwnd=0 отправитель приостанавливает обычную отправку новых данных и периодически выполняет контрольные пробы. Так он узнает, что окно снова открылось, даже если сообщение об изменении окна потерялось. Когда получатель объявит ненулевое окно, передачу можно продолжить с учётом cwnd и других ограничений.

Нулевое окно не означает разрыв соединения. Получатель сообщил, что пока не готов принимать новые данные. Если причина в том, что приложение не читает из сокета, более быстрый канал не поможет.

Размер окна передаётся в 16-битном поле Window заголовка TCP. Это окно приёма стороны, которая отправила данный сегмент. Если при установлении соединения стороны согласовали Window Scale, значение поля умножается на соответствующий коэффициент. Например, при сдвиге 3 значение 1000 означает $1000 \times 2^3 = 8000$ байт.

Для двух направлений коэффициенты могут различаться. Поля Window в самих SYN не масштабируются. Если запись трафика началась после установления соединения, в ней может не оказаться согласованного коэффициента. Подробнее: RFC 7323, §2.

Окно перегрузки: cwnd

Окно перегрузки, cwnd (congestion window), ограничивает объём отправленных данных, которые ещё не подтверждены. Их называют **данными в полёте**. Часть этих данных уже может находиться у получателя: пока ACK идёт обратно, отправитель продолжает считать их неподтверждёнными.

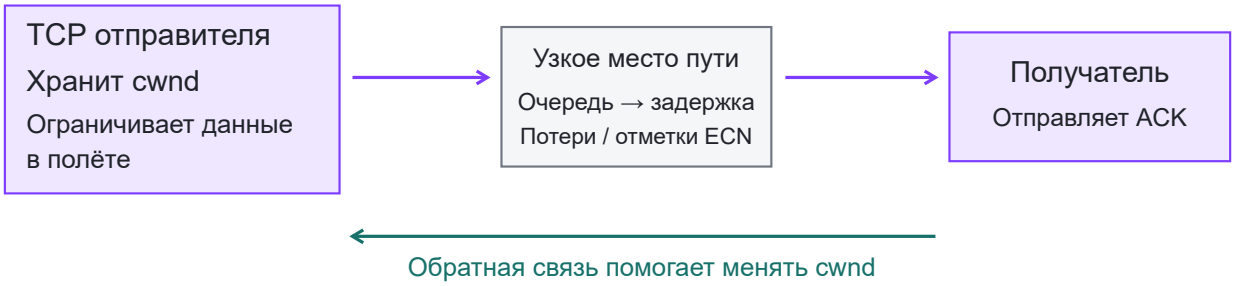


Рисунок 3. Отправитель оценивает состояние сети и изменяет cwnd.

cwnd хранится у отправителя и отсутствует в TCP-заголовке. Отправитель получает ACK, измеряет время и скорость доставки, замечает возможные потери. Алгоритм управления перегрузкой использует эти сведения, чтобы увеличивать или уменьшать cwnd. Узнать точное свободное место во всех очередях сети отправитель не может.

При поддержке ECN — Explicit Congestion Notification — сетевой узел может пометить пакет, чтобы сообщить о перегрузке. Получатель передаёт этот сигнал отправителю. Это позволяет уменьшить нагрузку ещё до потери пакетов. Механизм описан в RFC 3168. Сами потери также учитываются, но у них могут быть и другие причины.

Как два окна ограничивают отправку

Для расчёта примем простые условия: потерь нет, окна не меняются, у приложения есть готовые данные, один блок равен 1000 байтам. При rwnd=6000 и cwnd=4000 байт отправитель может держать в полёте не больше 4000 байт. Если 3000 байт уже отправлены и не подтверждены, можно отправить ещё 1000 байт.

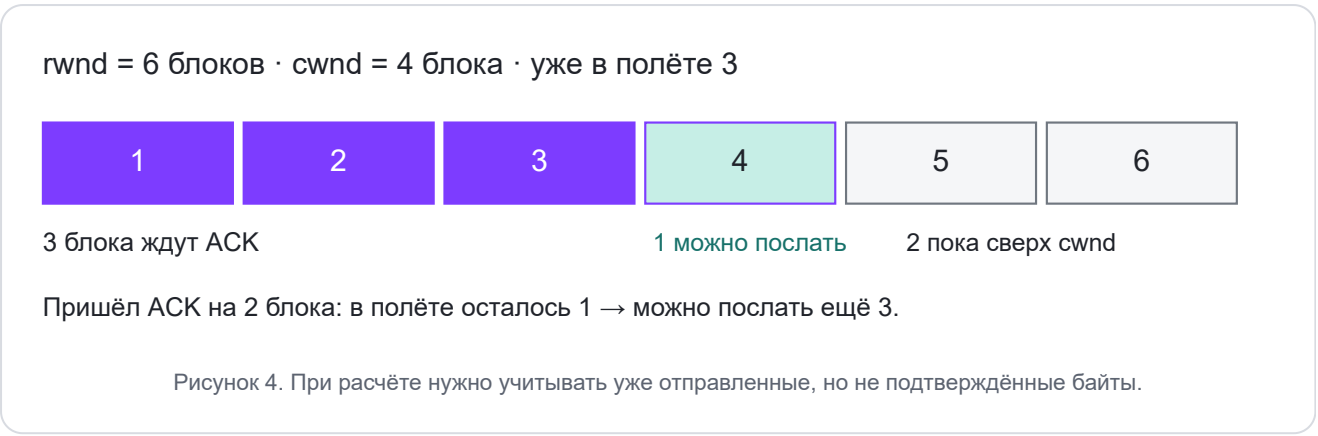


Рисунок 4. При расчёте нужно учитывать уже отправленные, но не подтверждённые байты.

```
Максимальный объём в полёте = min(rwnd, cwnd)
Можно отправить новых данных = max(0, min(rwnd, cwnd) − байты в полёте)

min(6000, 4000) − 3000 = 1000 байт
```

Теперь пришёл ACK, подтверждающий два из трёх блоков. Если ничего нового ещё не отправлено, в полёте остался один блок. При прежних размерах окон можно отправить ещё три блока. В реальном обмене новый ACK может также сообщить другое rwnd или привести к изменению cwnd.

Рассмотрим другой случай: rwnd=2000, cwnd=8000, в полёте 1000 байт. Отправить можно ещё 1000 байт, но теперь предел задаёт получатель. Увеличение cwnd не поможет. Байты, которые TCP подтвердил, а приложение ещё не прочитало, получатель уже учёл в объявленном rwnd. Вычитать их из доступного объёма повторно не нужно.

Эта формула описывает выбранные условия. При потерях и изменении окон работают дополнительные правила. Кроме того, разрешённые данные не обязательно отправляются сразу: механизм pacing распределяет отправку во времени, а у приложения может не быть готовых данных. Определения: RFC 5681, §2.

Как выбирают cwnd: Reno, CUBIC и BBR

Управлять перегрузкой в TCP можно с помощью разных алгоритмов. На классическом Reno удобно разобрать две фазы роста окна. В фазе slow start при достаточном количестве данных и подтверждений окно растёт примерно экспоненциально: условно 1, 2, 4, 8 единиц за последовательные RTT.

После достижения порога ssthresh (slow start threshold) начинается congestion avoidance. В учебной модели окно увеличивается примерно на один MSS за RTT: после 8 получаем 9, 10, 11. Числа показывают характер роста; реальное начальное окно не обязано равняться одному сегменту. Также окно не удваивается на каждый ACK: результат зависит от того, как приходят подтверждения и сколько данных отправляет приложение.

Алгоритм	На что опирается	Как регулирует передачу
Reno	Подтверждения и обнаружение потерь	После начального разгона постепенно увеличивает окно; при потерях уменьшает нагрузку
CUBIC	Подтверждения, признаки перегрузки и время после уменьшения окна	Использует кубическую функцию роста окна; подходит в том числе для быстрых соединений с большим RTT
BBR	Оценки скорости доставки, минимального RTT и другие сигналы	Управляет темпом отправки и объёмом данных в полёте на основе модели сети

В Reno тайм-аут и повторные ACK приводят к разным действиям, поэтому правило «при любой потере разделить окно пополам» слишком упрощает алгоритм. У BBR детали реакции также зависят от версии. Для разбора конкретного соединения нужно знать алгоритм и его реализацию в ОС. Ограничение rwnd продолжает действовать при любом из этих вариантов.

Источники: Reno, RFC 5681, CUBIC, RFC 9438, рабочая спецификация BBR. При чтении материалов о BBR обращайте внимание на версию алгоритма.

Какой алгоритм выбрать на практике

Для обычного сервиса разумно начать с алгоритма, выбранного ОС, и менять его после измерений. Ориентиры для выбора такие:

Алгоритм	Когда его рассматривают
Reno	Для обучения, сравнительных экспериментов и воспроизведения поведения старых систем. На быстрых соединениях с большим RTT он может слишком медленно наращивать окно
CUBIC	Как универсальный вариант для обычного интернет-трафика, в том числе длительных передач по быстрым каналам с большим RTT
BBR	Когда хотят проверить, улучшит ли оценка скорости доставки и RTT работу на путях с большими задержками, очередями или потерями, не связанными с перегрузкой

Выбор BBR стоит проверять на своей нагрузке: сравнивать скорость, задержку, потери и влияние на соседние потоки. Результат зависит от версии BBR и условий сети; гарантированного выигрыша во всех случаях нет. Материалы разработчиков: проект BBR.

Linux. В стандартной конфигурации основного дерева ядра и во многих дистрибутивах по умолчанию используется CUBIC. Дистрибутив или администратор может выбрать другой алгоритм. Текущую настройку и список доступных алгоритмов можно посмотреть без изменения системы:

```
sysctl net.ipv4.tcp_congestion_control
sysctl net.ipv4.tcp_available_congestion_control
```

Приложение также может выбрать алгоритм для отдельного сокета, поэтому системное значение не обязательно описывает каждое соединение. Основание: конфигурация ядра Linux и документация TCP sysctl.

Windows. Для обычных интернет-соединений в современных Windows 10/11 и Windows Server 2019 и новее типичный выбор по умолчанию — CUBIC. Microsoft описывает переход на него в обзоре TCP для Windows и обзоре Windows Server 2019. В Windows параметры задаются по профилям: Internet, Datacenter и другим. В старых версиях и специальных профилях встречаются другие алгоритмы, поэтому одного названия ОС недостаточно. Настройки профилей показывает PowerShell:

```
Get-NetTCPSetting | Select-Object SettingName, CongestionProvider
```

Эта команда показывает настройки, а не меняет их. Описание: Get-NetTCPSetting. Алгоритм управляет отправкой своей стороны; у двух участников соединения он может различаться.

RTT, объём данных в полёте и скорость

Представим, что после каждых 10 000 байт отправитель ждёт подтверждение 100 мс. Даже без потерь он сможет передавать не больше примерно 100 000 байт/с, или 0,8 Мбит/с. Быстрый канал сам по себе не устранит это ожидание. Чтобы использовать его пропускную способность, нужно отправлять следующие данные, пока предыдущие ещё не подтверждены.

Оценить необходимый объём помогает BDP — bandwidth-delay product, произведение пропускной способности на RTT. Для пути с пропускной способностью 100 Мбит/с и RTT 0,1 с получаем:

```
100 000 000 бит/с × 0,1 с = 10 000 000 бит = 1 250 000 байт = 1,25 МБ
```

Чтобы постоянно использовать такую пропускную способность, нужно держать в полёте сопоставимый объём данных. Если одно из окон разрешает лишь 64 000 байт, оценка скорости составит 64 000 / 0,1 = 640 000 байт/с, или 5,12 Мбит/с. В этих расчётах мы не учитываем потери, накладные расходы, конкурирующий трафик и ограничения приложения.

BDP даёт оценку объёма данных. Алгоритм управления перегрузкой решает, как изменять нагрузку при реальном обмене. Например, BBR использует оценки скорости доставки и времени распространения; при разборе Reno и CUBIC величина BDP тоже помогает понять поведение соединения.

Полученные 1,25 МБ не следует автоматически выставлять во всех буферах. Если измеренный RTT уже включает долгое ожидание в очередях, увеличение буферов может сохранить эти очереди и высокую задержку.

Как различить причины медленной передачи

Наблюдение	Возможная причина	Что проверить
Окно получателя падает до нуля	Приложение не успевает освобождать буфер	Читает ли приложение данные и что его задерживает
Окно получателя открыто, RTT велик, данных в полёте мало	Мало разрешает cwnd либо приложение редко отправляет данные	Размер cwnd и наличие готовых данных у отправителя
Есть повторные передачи и растёт задержка	Потери, задержки или перестановки; возможна перегрузка	Записи трафика на обеих сторонах и показатели сети

Эти наблюдения помогают выбрать следующую проверку. Например, при нулевом rwnd полезнее выяснить, почему приложение не читает данные, чем сразу менять размер сетевых буферов.

9. Поля и дополнительные механизмы TCP — справочник

Для чего нужны поля заголовка

К схеме заголовка из раздела 4 удобно возвращаться при чтении пакетов. Каждая группа полей отвечает на свой вопрос:

Поля	Что по ним можно узнать
Source Port, Destination Port	Между какими портами идёт обмен?
Sequence Number, Acknowledgment Number	Какие байты отправлены и подтверждены?
Data Offset	Где заканчивается заголовок и начинаются данные?
Флаги	Устанавливается или закрывается соединение? Есть ли подтверждение, сброс или сигнал ECN?
Window	Какое окно приёма объявляет сторона, отправившая сегмент?
Checksum	Обнаружены ли ошибки при проверке контрольной суммы?
Опции	Какие дополнительные параметры переданы: MSS, SACK, Window Scale, Timestamps и другие?

Порты занимают по 16 бит, SEQ и ACK — по 32. Нумерация байтов идёт по модулю 2^{32}: после максимального значения счёт продолжается с нуля. TCP учитывает это при сравнении номеров. В наших числовых примерах перехода через эту границу нет.

Флаги и длина данных

Флаг базового формата	Значение
SYN	Согласование начальных номеров при установлении соединения
ACK	Поле номера подтверждения нужно учитывать
FIN	Отправитель закончил передавать данные в своём направлении
RST	Сброс соединения или отказ в его установлении
PSH	Указание, связанное с передачей накопленных данных приложению; границ сообщений не задаёт
URG	Поле Urgent Pointer нужно учитывать
ECE, CWR	Флаги для передачи сигналов ECN в базовой схеме RFC 3168

В одном сегменте могут быть установлены несколько флагов, например SYN+ACK или FIN+ACK. Их смысл нужно рассматривать вместе с этапом обмена. В таблице показан базовый формат из RFC 9293; расширения могут добавлять свои правила.

PSH не обозначает конец сообщения приложения. URG также не задаёт общий приоритет пакета: он относится к специальному механизму срочных данных TCP. Если приложение использует этот механизм, его поведение нужно сверять со спецификацией и socket API конкретной ОС.

Контрольная сумма TCP учитывает заголовок, данные и псевдозаголовок IP. Она помогает обнаружить ошибки при передаче, но не проверяет подлинность отправителя и не защищает от намеренной подделки данных.

Отдельного поля длины данных TCP нет. Анализатор рассчитывает длину из размеров IP-пакета и заголовков. Поэтому `LEN=500` в наших примерах означает 500 байт данных, а не значение ещё одного поля TCP-заголовка.

Nagle и delayed ACK

Алгоритм Nagle помогает сократить число маленьких сегментов. Если ранее отправленные данные ещё не подтверждены, он может задержать небольшую новую порцию, чтобы отправить её вместе с другими данными.

Это одна из причин, почему две небольшие порции, переданные приложением отдельно, могут попасть в один TCP-сегмент и быть прочитаны получателем одним блоком — как в примере с `PING` и `PONG` в разделе 4. Но и без Nagle один вызов чтения может вернуть данные нескольких отправок: TCP не сохраняет их границы, а приложение читает уже накопившиеся байты.

Delayed ACK работает на стороне получателя: тот может ненадолго отложить ACK, например чтобы подтвердить несколько сегментов одним сообщением. Когда приложение обменивается небольшими запросами и ответами, совместная работа этих механизмов иногда приводит к заметным паузам.

Опция TCP_NODELAY отключает Nagle. Применять её стоит после проверки причины задержки: одинаковые паузы могут возникать и по другим причинам. На границы сообщений приложения эта опция не влияет.

Почему запись трафика на хосте может отличаться от пакетов в сети

Часть работы с пакетами ОС и сетевая карта выполняют совместно. Перенос таких операций называют offload. Например, TSO — TCP Segmentation Offload — позволяет разделить большой блок на сегменты позже, уже после того, как его увидела программа записи трафика на отправителе.

На приёме GRO — Generic Receive Offload — может объединять данные нескольких пакетов до того, как их увидит программа анализа. Поэтому большой сегмент в Wireshark на хосте не обязательно передавался по сети именно в таком виде.

Аналогично, в записи исходящего трафика контрольная сумма может выглядеть неверной, если сетевая карта должна вычислить её позже. Перед выводом о повреждении пакетов нужно проверить настройки offload, место записи и по возможности сравнить трафик с записью на принимающей стороне.

Пометка `retransmission` тоже является выводом анализатора по доступным пакетам. Причину повторной передачи она не устанавливает. Подробнее о механизмах: документация ядра Linux по offload.

10. SCTP и DCCP

TCP и UDP предлагают разные наборы свойств: поток байтов с восстановлением потерь и отдельные датаграммы без него. Другие транспортные протоколы сочетают эти свойства иначе.

SCTP

SCTP — Stream Control Transmission Protocol — надёжно передаёт сообщения и сохраняет их границы. Обмен между сторонами называется **ассоциацией**. Внутри ассоциации можно организовать несколько потоков и сохранять порядок сообщений отдельно в каждом из них. Это позволяет избежать части задержек, которые возникают, когда все сообщения идут в одном упорядоченном потоке.

Стороны ассоциации могут использовать несколько IP-адресов. Эта возможность, multihoming, позволяет переключиться на другой путь при отказе основного. Обнаружение отказа и переключение занимают время. Само наличие нескольких адресов не означает, что пропускная способность всех интерфейсов складывается.

При установлении ассоциации используется cookie-механизм. Он позволяет серверу отложить выделение ресурсов для ассоциации до получения корректного ответа от другой стороны.

SCTP разрабатывался в том числе для передачи телефонной сигнализации по IP. Ещё один пример применения — WebRTC data channels, где SCTP работает поверх DTLS. Потоки одной ассоциации всё равно используют общие ресурсы, а их взаимодействие зависит в том числе от правил фрагментации сообщений. Поэтому несколько потоков не устраняют все возможные задержки. Базовая спецификация: RFC 9260.

DCCP и трудности внедрения

DCCP — Datagram Congestion Control Protocol — передаёт датаграммы и управляет перегрузкой, но не повторяет потерянные пользовательские данные. Стороны устанавливают соединение и согласуют механизм управления перегрузкой. Такое сочетание полезно для приложений, которым важнее своевременная доставка, чем получение каждого сообщения. Устройство протокола описано в RFC 4340.

Одним из существенных препятствий для широкого внедрения стала совместимость с существующей инфраструктурой. Поддержки DCCP на клиенте и сервере недостаточно, если между ними есть NAT или межсетевой экран, который не умеет обрабатывать этот протокол. Обычному IP-маршрутизатору не обязательно разбирать поля DCCP; трудности прежде всего возникают у устройств, которые фильтруют трафик или ведут таблицы трансляции адресов и портов.

Для обхода этого ограничения появился способ передавать DCCP внутри UDP. RFC 6773 описывает как сам механизм, так и проблему NAT/NAPT без поддержки DCCP. Однако такой способ тоже требует реализации на обеих сторонах. В результате технические свойства протокола и возможность использовать его в уже работающей сети оказались двумя отдельными задачами.

Протокол	Что получает приложение	Восстановление потерь	Управление перегрузкой
TCP	Поток байтов	Да, пока соединение удаётся поддерживать	Да
UDP	Датаграммы	В самом протоколе нет	В самом протоколе нет
SCTP	Сообщения в потоках	Да, в базовом надёжном режиме	Да
DCCP	Датаграммы	Потерянные пользовательские данные не повторяются	Да

Таблица описывает базовые свойства. Например, у SCTP есть расширения, которые позволяют менять условия надёжной доставки.

11. RPC и удалённые операции — справочник

Что происходит при удалённом вызове

Транспорт доставляет данные между приложениями. Теперь рассмотрим, как приложения используют эти данные для совместной работы. Начнём с RPC — Remote Procedure Call, удалённого вызова процедуры.

В коде такой вызов может выглядеть как обычный: `result = service.reserve(item)`. Но запрос нужно передать по сети, выполнить на сервере и вернуть ответ. На любом из этих этапов возможен сбой.

```
Клиентский код
  → локальный stub / proxy
  → сериализация аргументов и идентификатора вызова
  → передача запроса
  → обработка запроса и вызов процедуры на сервере
  → сериализация результата или ошибки
  → передача ответа и возврат результата клиентскому коду
```

Stub — локальный объект или функция, через которые клиент обращается к удалённому сервису. Библиотека RPC берёт на себя сетевой обмен, сопоставляет ответы с вызовами, отслеживает тайм-ауты и обрабатывает ошибки. Сериализация превращает аргументы и результат в данные, которые можно передать по сети. Точное распределение работы между сгенерированным кодом и библиотекой зависит от RPC-системы.

Что подтверждает каждый этап

Событие	Что уже произошло	Чего мы ещё не знаем
Приложение передало данные сетевому стеку ОС	ОС приняла байты для отправки	Доставлены ли они другой стороне?
Пришёл TCP ACK на байты запроса	TCP получателя принял эти байты	Прочитало ли их серверное приложение?
Пришёл RPC-ответ	Сервер вернул результат или ошибку вызова	Что именно означает ответ по правилам операции?
Приложение сообщило об успехе	Операция считается выполненной по правилам сервиса	Включает ли успех сохранение результата и выполнение действий в других системах?

Представим, что сервер зарезервировал товар, но соединение оборвалось до получения ответа клиентом. Клиент может увидеть ту же ошибку, что и при обрыве до начала обработки. Если он просто повторит запрос, сервер может выполнить вторую операцию резервирования.

Операцию называют **идемпотентной**, если её повторное выполнение даёт тот же итоговый эффект, что и однократное. Например, повторная установка статуса «оплачено» сохраняет этот статус, а повторное списание денег приводит к ещё одному списанию. При этом ответы на первый и повторный вызовы могут различаться.

Чтобы распознавать повторные запросы, сервис может хранить идентификатор операции и её результат. Наличие ID в сообщении само по себе недостаточно: сервер должен использовать его для поиска уже выполненной операции. Надёжность TCP не снимает с приложения задачу обработки таких повторов.

ONC/Sun RPC и NFS

ONC — Open Network Computing. В ONC RPC клиент указывает программу, её версию и процедуру. Идентификатор Transaction ID помогает сопоставить ответ с вызовом, а XDR — External Data Representation — задаёт представление данных.

При передаче через TCP протокол добавляет правила определения границ RPC-записей: в самом TCP есть только поток байтов. Возможен и обмен через UDP; тогда особенно важны правила ожидания ответа и повторной отправки. Описание ONC RPC: RFC 5531.

NFS — Network File System — использует RPC для работы с файлами на другом узле. Для приложения чтение такого файла похоже на локальное, но за ним стоит сетевой обмен:

```
Приложение → файловый API → NFS-клиент → RPC → NFS-сервер → хранилище
```

Если чтение зависло, причиной могут быть сеть, удалённый сервер, блокировка файла или настройки клиента. Поэтому сначала полезно выяснить, где хранится файл и какая версия NFS используется.

Версии NFS различаются правилами хранения состояния, блокировками и набором операций. При диагностике нужны версия, транспорт, настройки монтирования, время ожидания и ответы сервера. Для знакомства с NFSv4.0 можно использовать RFC 7530.

Java RMI

RMI — Remote Method Invocation — позволяет вызывать методы удалённых объектов Java. Клиент получает ссылку на объект и обращается к методам его интерфейса. Локальный представитель передаёт вызов удалённой JVM.

Служба registry помогает получить первоначальную ссылку на объект:

```
Клиент → registry: получить ссылку на сервис
Клиент → адрес удалённого объекта: вызвать метод
```

После получения ссылки вызовы идут к удалённому объекту, поэтому доступность registry ещё не гарантирует доступность самого сервиса. В конфигурации нужно проверить адрес и порты, которые сервер сообщает клиенту. Они могут быть фиксированными или выбираться динамически — в зависимости от настройки.

RMI опирается на объектную модель Java. При взаимодействии важны совместимость типов и правила сериализации. Архитектура описана в Java RMI System Overview.

XML-RPC

В XML-RPC имя метода и параметры записываются в XML. Обычно запрос передаётся в теле HTTP POST, а ответ содержит результат или ошибку fault. Пример тела запроса:

```
<methodCall>
  <methodName>catalog.count</methodName>
  <params>
    <param><value><string>books</string></value></param>
  </params>
</methodCall>
```

Здесь HTTP передаёт сообщение, XML задаёт его структуру, а метод `catalog.count` определяет выполняемую операцию. Успешный HTTP-ответ может содержать ошибку XML-RPC, поэтому нужно проверять и HTTP-статус, и тело ответа. Формат сообщений описан в XML-RPC Specification.

SOAP

SOAP задаёт правила обмена структурированными сообщениями. В SOAP 1.2 сообщение содержит Envelope, необязательный Header и Body. Ошибка может передаваться как Fault. SOAP используют в том числе для RPC, но его возможности не ограничиваются вызовами процедур. Сообщения часто передают через HTTP, хотя модель допускает другие способы.

Пример тела SOAP 1.2:

```
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
  <env:Body>
    <m:Count xmlns:m="urn:course:catalog">
      <m:Category>books</m:Category>
    </m:Count>
  </env:Body>
</env:Envelope>
```

При разборе сообщения нужно учитывать версию SOAP, пространства имён XML, структуру Body, обязательные заголовки и возможный Fault. Эти правила отличают SOAP от произвольного XML-сообщения в HTTP.

WSDL — Web Services Description Language — позволяет описать интерфейс сервиса и способы обращения к нему. По этому описанию можно подготовить клиент, а сами вызовы передаются отдельными сообщениями. Спецификация SOAP: SOAP 1.2 Part 1.

JSON-RPC

В запросе JSON-RPC 2.0 указываются метод, параметры и идентификатор вызова. Например:

```
{"jsonrpc":"2.0","method":"catalog.count","params":{"category":"books"},"id":17}
{"jsonrpc":"2.0","result":42,"id":17}
```

Поле `id` позволяет сопоставить ответ с запросом. Оно не делает операцию идемпотентной. Ответ содержит результат `result` или ошибку `error`. Запрос без `id` называется notification — уведомлением, на которое сервер не отправляет ответ JSON-RPC.

JSON-RPC не требует использовать HTTP. При передаче через TCP нужно определить, как получатель будет находить границы сообщений в потоке байтов; это называют framing. Если выбран HTTP, нужно учитывать его статусы и правила обмена.

Несколько запросов можно объединить в batch. Однако такое объединение не делает их одной транзакцией, а порядок ответов может отличаться от порядка запросов. Сопоставлять их следует по `id`.

При диагностике полезно последовательно проверить: пришли ли данные, удалось ли разобрать JSON, соответствует ли сообщение JSON-RPC и не вернул ли удалённый метод ошибку. Спецификация: JSON-RPC 2.0.

gRPC

В gRPC разработчик описывает сервис и сообщения, после чего инструменты генерируют клиентский и серверный код. Обычно используются Protocol Buffers и HTTP/2. Например, интерфейс подсчёта товаров можно описать так:

```
service Catalog {
  rpc Count (CountRequest) returns (CountReply);
}
message CountRequest { string category = 1; }
message CountReply   { int64 count = 1; }
```

gRPC поддерживает несколько вариантов обмена:

- **Unary:** один запрос и один ответ.
- **Server streaming:** один запрос и последовательность ответных сообщений.
- **Client streaming:** последовательность сообщений от клиента и один ответ.
- **Bidirectional streaming:** обе стороны передают последовательности сообщений.

Поток gRPC описывает обмен сообщениями приложения. Поток HTTP/2 переносит его данные внутри HTTP/2-соединения, а TCP передаёт байты всего соединения. Поэтому потеря в TCP может задержать несколько вызовов gRPC одновременно.

У вызова есть метаданные, сообщения и итоговый статус. HTTP 200 не заменяет проверку статуса gRPC. Параметр deadline задаёт срок, до которого клиент готов ждать завершения вызова. Если клиент отменил вызов или дождался этого срока, сервер уже мог успеть выполнить операцию.

Для поиска проблемы нужны имя метода, deadline, итоговый статус, этап обмена, доступность адреса сервиса и настройки прокси. Описание: gRPC Core Concepts.

Сравнение семейств RPC

Семейство	Основная идея	Что проверить при сбое
ONC RPC / NFS	Вызовы процедур, XDR, файловые операции	Версии, транспорт, состояние клиента и сервера
Java RMI	Вызов методов удалённых объектов Java	Доступность registry и самого объекта, адреса, порты, типы
XML-RPC	Метод и параметры в XML	HTTP-статус и наличие fault в ответе
SOAP	Структурированные сообщения с расширяемой обработкой	Версию, структуру сообщения, заголовки и Fault
JSON-RPC	Метод, параметры, ID и результат или ошибка	Границы сообщений, соответствие ID, правила уведомлений и повторов
gRPC	Описание сервисов, отдельные вызовы и потоки сообщений	Deadline, статус gRPC, прокси и транспорт

Выбор семейства зависит в том числе от совместимости с уже работающими системами. Для нашей темы во всех вариантах важно одно: доставка байтов, получение ответа и выполнение удалённой операции — отдельные этапы, которые нужно проверять по отдельности.

12. Сигнализация и передача звука — справочник

Как организован телефонный вызов

Во время звонка приложения решают несколько задач: находят абонента, предлагают ему принять вызов, согласуют параметры и передают звук. Для этого используются разные протоколы:

Протокол	Задача
SIP, Session Initiation Protocol	Установить, изменить или завершить сеанс связи
SDP, Session Description Protocol	Описать адреса и параметры аудио- и видеопотоков
RTP, Real-time Transport Protocol	Передавать аудио и видео с номерами пакетов и временными метками
RTCP, RTP Control Protocol	Передавать отчёты об участниках и качестве приёма

Эти задачи связаны, но выполняются отдельно. Поэтому успешное установление вызова ещё не гарантирует, что собеседники услышат друг друга. Также название протокола не определяет его место в OSI: например, RTP обычно работает поверх UDP, хотя в его названии есть слово Transport.

Участники SIP

Телефон или программа для звонков выступают как User Agent. В конкретном обмене запросом и ответом User Agent играет роль клиента, если отправляет запрос, и сервера, если отвечает на него. SIP proxy пересылает запросы, а registrar принимает регистрацию, которая связывает пользователя с адресом для связи с ним. Один продукт может выполнять сразу несколько ролей.

```
Абонент A → SIP proxy → абонент B      сигнальные сообщения
Абонент A ⇔ media relay ⇔ абонент B    один из вариантов передачи звука
```

Звук может идти прямо между абонентами или через промежуточный узел — media relay. Адреса для передачи RTP могут отличаться от адреса SIP proxy, поэтому их доступность нужно проверять отдельно.

SIP может использовать UDP или TCP. TLS позволяет защитить сигнальные сообщения при соответствующем способе передачи, но сам звук идёт отдельно. Для защиты аудио- и видеоданных применяются другие механизмы, например SRTP.

Пример SIP-обмена

Ниже показан упрощённый вызов между двумя участниками, без промежуточных узлов:

```
A → B: INVITE  [предложение SDP]
B → A: 180 Ringing
B → A: 200 OK  [ответ SDP]
A → B: ACK
A ↔ B: медиапотоки по согласованным параметрам
A → B: BYE
B → A: 200 OK
```

`INVITE` предлагает начать сеанс, `180 Ringing` сообщает о вызове абонента, а `200 OK` в этом обмене означает, что приглашение принято. Затем A подтверждает ответ сообщением SIP ACK. Для завершения установленного диалога используется `BYE`.

Метод `CANCEL` нужен для отмены ещё не завершённого запроса, например когда вызывающий отменяет звонок до ответа. `REGISTER` используется для регистрации контактного адреса пользователя и в этой схеме не показан.

SIP ACK подтверждает событие в SIP-обмене, тогда как TCP ACK подтверждает получение байтов. Аналогично, `200 OK` на схеме — статус SIP, хотя такое же написание знакомо нам по HTTP. Правила обмена описаны в RFC 3261.

SDP: какие параметры согласуют стороны

В SDP стороны описывают, куда отправлять аудио или видео и какие форматы использовать. Например, предложение одного аудиопотока может выглядеть так:

```
v=0
o=alice 1000 1 IN IP4 192.0.2.10
s=Course call
c=IN IP4 192.0.2.10
t=0 0
m=audio 40000 RTP/AVP 0
a=rtpmap:0 PCMU/8000
```

Строка	Что означает в примере
o=	Кто создал описание, идентификатор и версия сеанса, адрес
c=	IP-адрес для передачи медиаданных
m=audio 40000…	Аудиопоток на порту 40000, профиль RTP/AVP, тип полезной нагрузки 0
a=rtpmap…	Для типа 0 используется PCMU с тактовой частотой 8000 Гц

Другая сторона отвечает параметрами, которые готова использовать. Предложение называют offer, ответ — answer. При диагностике их сравнивают: согласованы ли адреса, направления передачи, кодеки и типы полезной нагрузки (payload types).

Пример показывает базовое описание потока. Прохождение через NAT и защита медиаданных требуют дополнительных механизмов. SDP описан в RFC 8866, модель offer/answer — в RFC 3264.

RTP и RTCP

RTP добавляет к медиаданным номер последовательности (sequence number), временную метку (timestamp), идентификатор источника SSRC — Synchronization Source — и тип полезной нагрузки.

Номер последовательности помогает обнаружить пропуски и перестановки пакетов. Временная метка указывает положение данных на временной шкале аудио или видео. Она нужна для воспроизведения и сама по себе не показывает время суток. RTP не гарантирует доставку всех пакетов.

RTCP передаёт отчёты о приёме, в том числе о потерях и jitter. По ним участники могут оценивать качество связи и адаптировать передачу. Это периодические отчёты, а не обязательный ACK на каждый RTP-пакет. Для восстановления данных и адаптации качества система может использовать дополнительные механизмы. RTP и RTCP описаны в RFC 3550.

«Звонок установился, но звука нет»

Проверять такой случай удобно по этапам:

1. Завершился ли SIP-обмен и какие параметры SDP стороны приняли?
2. На какие адреса и порты каждая сторона отправляет RTP?
3. Видны ли пакеты на отправителе и приходят ли они к получателю?
4. Звука нет у обоих участников или только у одного?
5. Совпадают ли кодеки, типы полезной нагрузки, направления передачи и настройки защиты?

Если RTP не приходит, нужно проверить отправку, адреса, маршрутизацию и фильтрацию. Если пакеты приходят, следующий шаг — проверить, может ли приложение расшифровать, декодировать и воспроизвести данные. Так мы последовательно разбираем доставку, формат и обработку звука.

13. H.323, SIP и распределение функций — справочник

Что входит в H.323

Организовать звонок можно разными способами. Часть задач выполняют телефоны или программы на устройствах пользователей, часть — сетевые сервисы. Посмотрим на это распределение на примере H.323.

H.323 — семейство рекомендаций ITU-T для мультимедийной связи в пакетных сетях. Оно описывает несколько видов участников:

- **Терминал** — устройство или программа участника разговора.
- **Шлюз** — узел для связи с другими сетями.
- **Gatekeeper** — узел, который может обслуживать регистрацию, преобразование адресов и допуск к вызовам в своей зоне.
- **MCU, Multipoint Control Unit** — узел для организации конференций.

Сеть H.323 может работать без gatekeeper. Если он используется, часть управляющих действий проходит с его участием.

Рекомендация H.225.0 описывает сигнализацию вызова и RAS — Registration, Admission and Status: регистрацию, допуск и обмен сведениями о состоянии. H.245 используется для согласования возможностей участников и логических каналов. Управляющие сообщения описываются с помощью ASN.1 и бинарного кодирования — эти понятия мы встречали в лекции 2. Сам звук или видео могут передаваться по RTP.

```
Терминал ⇔ gatekeeper: регистрация и допуск, если gatekeeper используется
Терминал ⇔ другой терминал или сетевой узел: сигнализация вызова
Участники: согласование возможностей и каналов передачи аудио и видео
Участники: передача медиаданных по согласованным параметрам
```

Схема показывает основные задачи; конкретный порядок и участники обмена зависят от устройства сети. Обзор и связанные рекомендации: ITU-T H.323.

Какие задачи выполняют устройства и сетевые сервисы

И H.323, и SIP позволяют распределять работу между устройствами пользователей и сетевой инфраструктурой. Например, телефон может сам формировать запросы и обрабатывать ответы, а сетевой сервис — проверять право на звонок и выбирать, куда его направить.

Поэтому по одному названию протокола нельзя определить, насколько сложны телефон и сеть. Терминалы H.323 выполняют свою протокольную логику, а SIP-сеть может содержать много централизованных сервисов.

При изучении конкретной системы полезно выяснить, **кто находит абонента, разрешает вызов, согласует параметры и передаёт звук**. Такое описание показывает распределение функций лучше, чем общая характеристика «вся логика на устройствах» или «вся логика в сети».

IMS и SBC

IMS — IP Multimedia Subsystem — архитектура мультимедийных услуг, в которой сетевые сервисы управляют сеансами и доступом к услугам. Для сигнализации используется SIP. Например, функции CSCF — Call Session Control Function — участвуют в управлении вызовами и обращении к сервисам. Подробнее: спецификации IMS в 3GPP.

SBC — Session Border Controller — работает на границе сетей. Он может контролировать разрешённые соединения, обрабатывать сигнальные сообщения и помогать взаимодействовать системам с разными настройками. В зависимости от реализации через него могут проходить и медиаданные.

Если SBC изменяет сообщения или принимает и создаёт отдельные соединения, по разные стороны от него будут видны разные адреса, заголовки или соединения. Это нужно учитывать при сравнении записей трафика. Функции SBC и связанные с ними ограничения обсуждаются в RFC 5853.

14. Безопасность и переход к QUIC

Защита данных и доступность соединения

TCP не шифрует данные и не проверяет, что клиент подключился именно к нужному сервису. В HTTPS эти задачи решает TLS: при корректной проверке сертификата клиент подтверждает подлинность сервера, а обмен получает защиту от чтения и изменения данных посторонними.

Однако защищённый обмен тоже зависит от доставки пакетов. Потери, перегрузка или разрыв TCP-соединения могут помешать работе приложения, даже если с шифрованием всё в порядке.

Один из примеров атаки на доступность — SYN flood. Сервер получает много запросов на установление соединений и тратит ресурсы на их обработку и хранение сведений о незавершённых соединениях. Механизм SYN cookies позволяет отложить выделение части этих ресурсов до получения корректного ответа от клиента. Он помогает в определённых условиях, но не устраняет, например, перегрузку самого канала связи.

Непредсказуемые ISN и проверки входящих сегментов также затрудняют подделку TCP-пакетов атакующим, который не видит обмен. Подробный разбор атак и защиты оставим соответствующей лекции. Обзор SYN flood: RFC 4987.

Почему дальше QUIC

Мы увидели, что потеря одного участка TCP-потока может задержать несколько запросов HTTP/2. Установление TCP и согласование TLS — отдельные этапы, на которые тоже требуется время.

В следующей лекции рассмотрим другой подход — QUIC. Он работает поверх UDP и сам реализует необходимые транспортные механизмы. Разберём, как устроены его потоки, как устанавливается защищённое соединение и как оно может продолжаться при смене сетевого пути. Там же обсудим HTTP/3, ускоренное начало обмена 0-RTT и связанные с ним риски.

15. Самопроверка

1. Три клиента подключились к TCP-порту 443 одного сервера. Какие поля позволяют различить соединения? Почему одного порта сервера недостаточно?
2. Отправитель передал данные двумя вызовами отправки, а получатель прочитал их за три вызова чтения. Возможно ли это в TCP? Объясните почему.
3. Сервер получил байты 1001–1500 и 2001–2500, но ещё не получил 1501–2000. Какой ACK он отправит в нашей модели? Что изменится, когда пропуск заполнится?
4. TCP подтвердил все байты запроса, но RPC-ответ не пришёл. Что могло произойти с операцией на сервере?
5. Получатель объявил нулевое окно. Почему увеличение пропускной способности канала может не ускорить передачу?
6. Где хранится cwnd? Можно ли прочитать его из поля Window в TCP-пакете?
7. Какие события подтверждают TCP ACK, SIP ACK и RPC-ответ?
8. Почему соединение может прерваться, даже если данные защищены TLS?
9. SIP-вызов установлен, RTP-пакеты приходят, но звука нет. Что ещё нужно проверить?
10. Какой объём данных в полёте примерно нужен для использования пути с пропускной способностью 100 Мбит/с и RTT 100 мс? Чего не учитывает оценка?
11. rwnd=6000, cwnd=4000, в полёте 3000 байт. Сколько новых байтов можно отправить в нашей модели? Как изменится ответ после подтверждения 2000 байт, если окна прежние и новых отправок ещё не было?
12. Wireshark показывает SEQ=0 у SYN. Значит ли это, что ОС выбрала предсказуемый ISN?
13. Что позволяет оценить BDP и какую задачу решает алгоритм управления перегрузкой?
14. Найдите в схеме TCP-заголовка ISN, cwnd и LEN. Какие из этих обозначений соответствуют отдельным полям?

Ответы и пояснения

1. Нужны IP-адреса и порты обеих сторон; для различения транспортов добавляют протокол. Порт 443 у сервера одинаков для всех трёх соединений, но пары адресов и портов клиентов различаются.
2. Да. TCP сохраняет последовательность байтов, но не границы вызовов отправки. Размеры блоков, которые читает получатель, могут быть другими.
3. ACK=1501: это номер следующего недостающего байта. После получения 1501–2000 сервер отправит ACK=2501, если ранее пришедшие байты 2001–2500 сохранены.
4. Приложение могло ещё не прочитать запрос, выполнять его, завершить с ошибкой или успешно выполнить, но не доставить ответ клиенту. TCP ACK не позволяет различить эти варианты.
5. Получатель сейчас не готов принимать новые данные. Например, приложение не читает их из заполненного буфера. Более быстрый канал не освобождает этот буфер.
6. cwnd хранится у отправителя. Поле Window сообщает окно приёма той стороны, которая отправила пакет; значения cwnd в нём нет.
7. TCP ACK подтверждает получение байтов. В показанном SIP-обмене SIP ACK подтверждает успешный ответ на INVITE. RPC-ответ сообщает результат или ошибку удалённого вызова по правилам конкретного протокола.
8. TLS защищает содержимое обмена, но пакеты могут потеряться, сеть — стать недоступной, а узел — закрыть соединение.
9. Нужно проверить согласованные кодеки и типы полезной нагрузки, направления передачи, настройки защиты, декодирование и воспроизведение.
10. BDP составляет около 1,25 МБ в десятичных единицах. Оценка не учитывает потери, накладные расходы, конкурирующий трафик и ограничения приложения. Это не готовое значение для настройки всех буферов.
11. Сначала можно отправить 1000 байт. После ACK неподтверждёнными остаются 1000 байт, поэтому при прежних окнах можно отправить ещё 3000 байт.
12. Нет. Wireshark может показывать номер относительно ISN. Тогда SYN отображается с SEQ=0 независимо от реального начального номера.
13. BDP оценивает объём данных по пропускной способности и RTT. Алгоритм управления перегрузкой определяет, как менять нагрузку в зависимости от подтверждений, потерь и других сигналов.
14. Отдельных полей с этими названиями нет. ISN — начальное значение SEQ, cwnd хранится у отправителя, а длину данных LEN анализатор вычисляет по размерам пакета и заголовков.

16. Источники и материалы курса

Ссылки на спецификации приведены рядом с объяснениями механизмов. Для дальнейшего чтения можно начать с трёх документов:

- TCP, RFC 9293 — формат сегмента, установление и закрытие соединения, нумерация и обработка данных.
- UDP Usage Guidelines, RFC 8085 — рекомендации для приложений, которые используют UDP.
- SIP, RFC 3261 — участники, запросы, ответы и правила SIP-обмена.

Для повторения предыдущих тем: презентация лекции 1, лонгрид лекции 2, презентация лекции 3. Ссылки на материалы курса открывают PDF из той же папки.

Дополнительные материалы:

- RTP и RTCP, RFC 3550 — передача медиаданных и отчёты о приёме.
- WebRTC Data Channels, RFC 8831 — применение SCTP для каналов данных WebRTC.
- H.323 и связанные рекомендации — организация мультимедийной связи и роли её участников.

При разборе конкретной системы важно учитывать версию протокола и документацию её реализации: они определяют доступные возможности и настройки.